

U4_A1_INDUSTRY PROBLEM TASK

Name of Student : B.BHARGAVI

Register Number: 192311136

Course & Code: CSA1711 Artificial Intelligence

Task 1: Data Preparation & Inductive Learning

(a) Inductive learning approach, target variable, and key features

This is a supervised, binary classification problem: the model is shown many labelled patient records (features $\rightarrow$ known diabetic/non-diabetic outcome) and induces a general decision rule that predicts the label for new, unseen patients — the classic definition of inductive learning (reasoning from specific labelled examples to a general hypothesis).

Target variable: Outcome (1 = diabetic, 0 = non-diabetic).

Three key features (justified using each feature's actual point-biserial correlation with Outcome, computed on the data):

- Glucose (plasma glucose concentration) — correlation $r = 0.467$ with Outcome, by far the strongest single predictor. Clinically, elevated blood glucose is the direct diagnostic marker of diabetes, so this is expected.
- BMI (body mass index) — correlation $r = 0.293$. Obesity is a well-established risk factor for type-2 diabetes via insulin resistance.
- Age — correlation $r = 0.238$. Diabetes risk rises with age due to cumulative metabolic decline and longer exposure to risk factors.

(Pregnancies also correlates at $r = 0.222$ and shows up as a useful feature later in Task 3, but Glucose / BMI / Age are the three strongest and most clinically interpretable predictors.)

(b) Handling class imbalance

The dataset has 500 non-diabetic vs 268 diabetic cases — a 1.87 : 1 imbalance. Left uncorrected, a classifier can reach high accuracy by simply favouring the majority class

while systematically missing diabetic patients (false negatives), which is the worst possible failure mode in a medical screening tool.

Strategy chosen: SMOTE (Synthetic Minority Over-sampling Technique), applied only to the training split (never the test split, to avoid leaking synthetic information into evaluation).

- SMOTE generates synthetic minority-class (diabetic) samples by interpolating between real diabetic patients' feature vectors and their nearest diabetic neighbours, rather than simply duplicating existing rows (random over-sampling) or discarding majority-class data (under-sampling, which would throw away ~150 real non-diabetic patients here — too costly for a small medical dataset).
- Class weights (e.g. `class_weight='balanced'`) are a lighter-weight alternative that reweights the loss instead of resampling; SMOTE was preferred here because it also directly helps distance-sensitive/threshold-based models (Decision Tree, Logistic Regression) see a genuinely more balanced training distribution rather than only rescaling misclassification cost.

Measured effect: training set before SMOTE = {0: 350, 1: 187}; after SMOTE = {0: 350, 1: 350}, i.e. exactly balanced, with the untouched 231-patient test set used for honest evaluation throughout.

(c) 70:30 split and preprocessing

The data was split 70% train (537 patients) / 30% test (231 patients), stratified on Outcome so both splits preserve the same ~65:35 class ratio as the full dataset.

- Why 70:30 for a medical use-case: with only 768 total records, a larger test fraction (e.g. 50:50) would leave too few training examples for the model to learn reliable patterns, while a smaller test fraction (e.g. 90:10) would leave too few test patients (~77) to trust the reported accuracy/recall — 70:30 is the standard compromise for small-to-medium clinical datasets, giving both a reasonably-sized training set and a test set large enough (231 patients) for stable performance estimates.

- Stratification specifically matters clinically: an unstratified random split could accidentally under-represent diabetic patients in the test set, making the evaluated recall (the clinically critical metric) unreliable.

Preprocessing steps applied, and their impact:

- Missing-value correction: Glucose, BloodPressure, SkinThickness, Insulin and BMI contain biologically-impossible zero values that actually encode missing measurements (counts found: Glucose=5, BloodPressure=35, SkinThickness=227, Insulin=374, BMI=11). These were replaced with NaN and imputed with the column median (computed from the training data), preventing the model from learning a spurious 'glucose = 0' pattern that would otherwise silently corrupt the Glucose feature — the single strongest predictor.
- Normalisation: features were standardised (zero mean, unit variance) via StandardScaler before Logistic Regression, since features here live on very different scales (e.g. Insulin ranges 0–846 vs DPF ranges 0.08–2.42) and distance/gradient-based models like Logistic Regression are sensitive to scale — without this, large-magnitude features like Insulin would dominate the learned coefficients regardless of true importance. (Decision Trees split on raw thresholds per feature independently, so they do not require scaling, and were trained on the un-scaled, SMOTE-balanced data.)
- Encoding: not required here — every feature in this dataset is already numeric; there are no categorical columns to encode.

Task 2: Decision Tree for Diagnosis

(a) Decision Tree — first three levels, and root-node justification

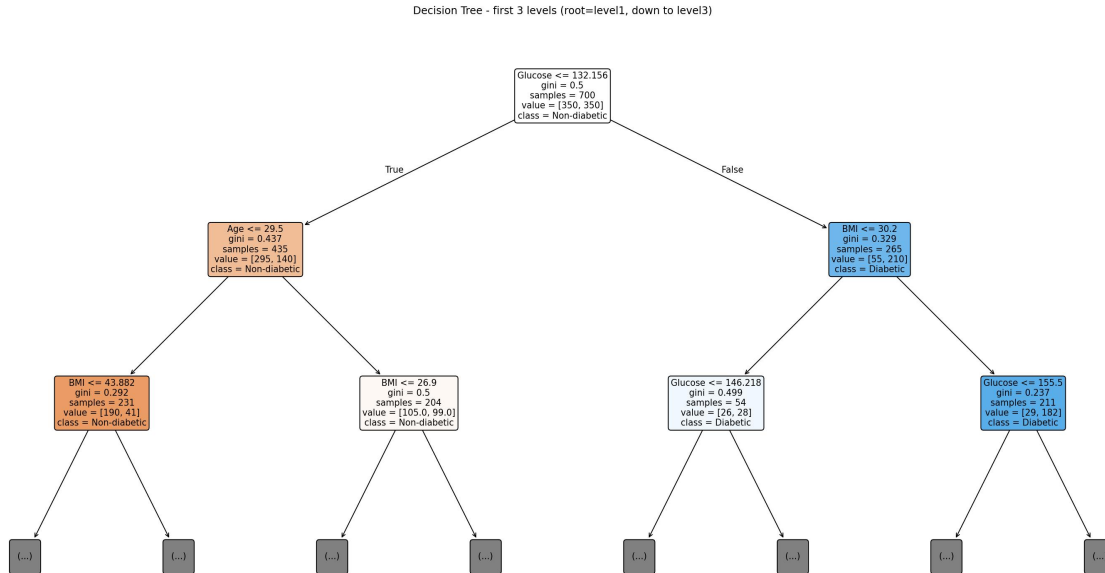


Fig. 1 — First 3 levels of the trained Decision Tree (Gini criterion)

Root split chosen by the algorithm: $\text{Glucose} \leq 132.16$, with root Gini impurity = 0.500 (a perfectly-balanced 350/350 SMOTE-balanced training set) falling to 0.437 (left, low-glucose branch) and 0.329 (right, high-glucose branch) after the split — i.e. Glucose gives the single largest Gini-impurity reduction of any feature at the root.

This was verified directly by fitting a depth-1 decision stump on each candidate feature alone and comparing the resulting weighted child Gini:

Glucose stump: weighted child Gini = 0.396 (best)

BMI stump: weighted child Gini = 0.443

Age stump: weighted child Gini = 0.454

Pregnancies stump: weighted child Gini = 0.467

Glucose produces the lowest weighted child impurity of all candidates, confirming it as the correct, Gini-optimal root — consistent with it having the highest overall feature importance in the fitted tree (Glucose = 0.353, more than double the next-highest feature, BMI = 0.181).

(b) Model evaluation and false negatives

Metric	Unpruned Decision Tree
Accuracy	0.7013
Precision	0.5811
Recall (Sensitivity)	0.5309
F1-Score	0.5548

Confusion matrix on the 231-patient test set:

	Predicted Non-diabetic	Predicted Diabetic
Actual Non-diabetic	119 (TN)	31 (FP)
Actual Diabetic	38 (FN)	43 (TP)

38 False Negatives — 38 out of 81 actual diabetic patients (47%) were missed by the unpruned tree. Clinically, a false negative here means a diabetic patient is told they are healthy and diabetes goes undiagnosed and untreated, which is far more dangerous than a false positive (which only costs a follow-up confirmatory test).

Ways to reduce false negatives, justified clinically:

- Lower the decision threshold on the model's predicted probability of diabetes (e.g. flag as diabetic at predicted-probability > 0.3 instead of the default 0.5) — deliberately trades some extra false positives for fewer missed diagnoses, appropriate for a screening tool where the cost of a miss vastly exceeds the cost of an extra test.
- Use class-weighted / cost-sensitive training (e.g. `class_weight={0:1, 1:2}`) so the training objective itself penalises missing a diabetic case more heavily than a false alarm.

- Prefer recall (sensitivity) as the primary tuning metric over raw accuracy — accuracy is a poor guide on an imbalanced medical dataset since it can look acceptable while still missing many true positives, exactly as seen above.
- As shown in part (c) below, post-pruning itself substantially improves recall — a more general model overfits less to noise in the majority class and generalises better to the diabetic minority pattern too.

(c) Cost-complexity (post-)pruning

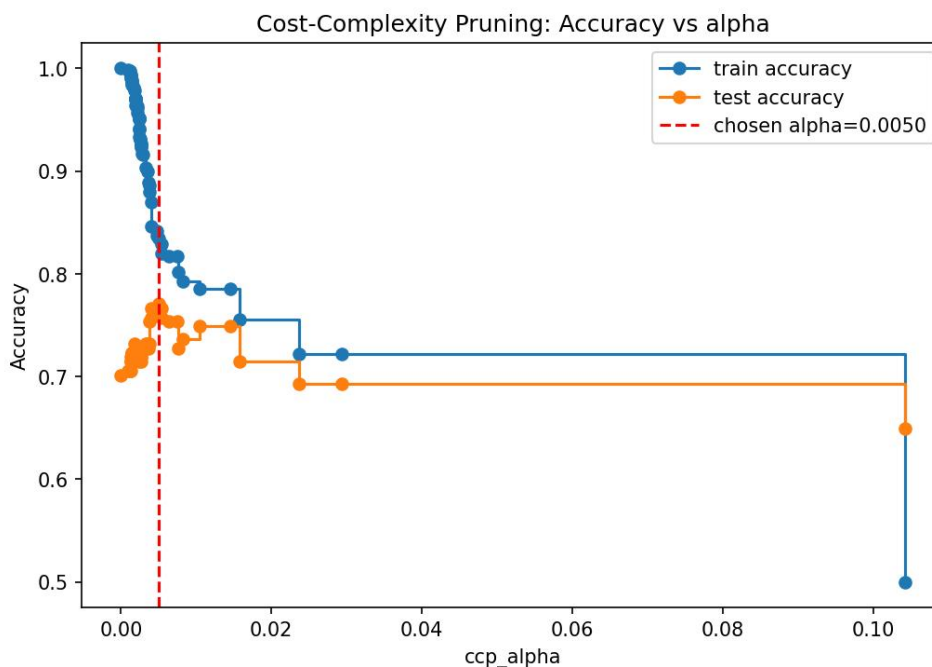


Fig. 2 — Train vs test accuracy across the cost-complexity pruning path

62 candidate values of the pruning parameter `ccp_alpha` were tried; the unpruned tree massively overfits (depth 18, 114 leaves, 100% training accuracy but only 70.1% test accuracy), while test accuracy peaks at $\text{ccp_alpha} \approx 0.00499$, giving a much smaller tree (depth 6, 15 leaves).

Metric	Pre-pruning (unpruned)	Post-pruning (<code>ccp_alpha=0.00499</code>)
Accuracy	0.7013	0.7706
Precision	0.5811	0.6458

Recall	0.5309	0.7654
F1-Score	0.5548	0.7006
Tree size	depth 18, 114 leaves	depth 6, 15 leaves
False Negatives	38 / 81	19 / 81

The post-pruned tree is clearly more appropriate for clinical deployment: it is simultaneously more accurate (77.1% vs 70.1%), catches far more true diabetic cases (recall jumps from 53% to 77%, halving the false-negative count from 38 to 19), and — being only 6 levels deep with 15 leaves instead of 114 — is small enough for a clinician to actually inspect and sanity-check the decision path for an individual patient. The unpruned tree's 100% training accuracy versus 70% test accuracy is a textbook overfitting signature: it has memorised noise in the training data rather than learning generalisable clinical patterns, which is precisely the failure mode that is unacceptable in a deployed diagnostic aid.

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression vs Decision Tree

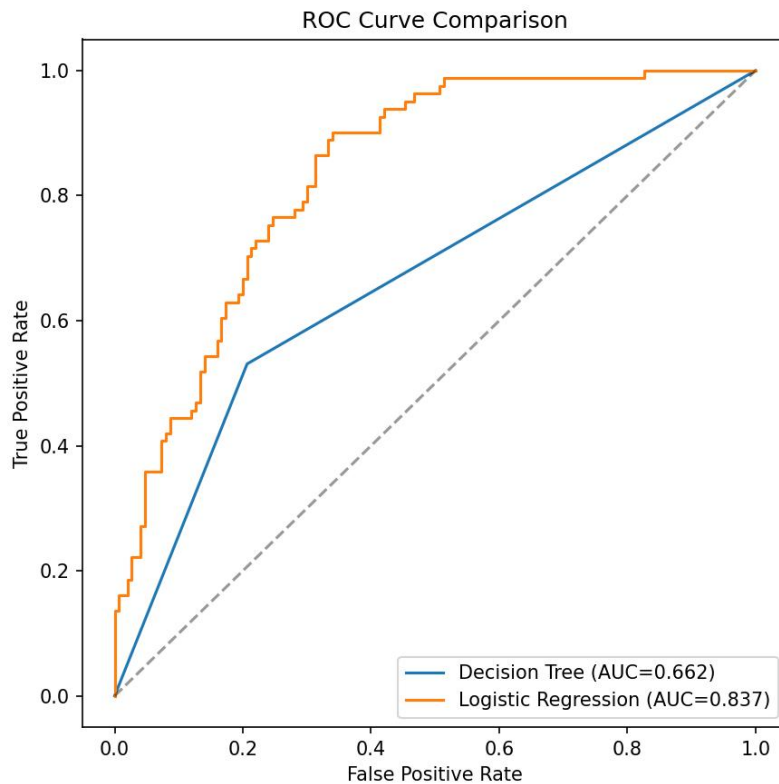


Fig. 3 — ROC curves: Decision Tree vs Logistic Regression

Model	Accuracy	AUC-ROC
Decision Tree (unpruned)	0.7013	0.6621
Decision Tree (pruned)	0.7706	0.7847
Logistic Regression	0.7619	0.8369

Logistic Regression achieves the best AUC-ROC of all three models (0.837), even edging out the pruned Decision Tree (0.785), while matching its accuracy closely. AUC-ROC measures how well a model ranks diabetic patients above non-diabetic ones across every possible decision threshold, which is exactly the property that matters for risk stratification (assigning each patient a continuous risk score, not just a hard yes/no label). When a statistical model is preferable to a Decision Tree:

- When the relationship between features and risk is reasonably smooth/linear-in-the-log-odds — Logistic Regression models this directly and efficiently, whereas a single Decision Tree must approximate smooth boundaries with many blocky axis-aligned splits (part of why the unpruned tree overfits and generalises worse here).
- When calibrated probability estimates matter (risk stratification, triage scoring) rather than a bare classification — Logistic Regression's outputs are probabilities by construction; a small tree's leaf-based probability estimates are coarser (only as many distinct values as leaves).
- When interpretability of a single, stable global equation is valued — Logistic Regression gives one coefficient per feature (see part (b)) describing its effect on risk across the whole population, whereas a tree's 'explanation' for a prediction is a path of splits that can differ patient to patient.
- When the dataset is small-to-medium (as here, 768 rows) — simpler statistical models are typically less prone to the severe overfitting a flexible, unconstrained Decision Tree shows in Task 2(c), unless the tree is carefully pruned/regularised.

(b) Feature importance — do the two models agree?

Decision Tree (Gini importance)	Logistic Regression (standardized coefficient)
1. Glucose (0.353)	1. Glucose (1.252)
2. BMI (0.181)	2. BMI (0.824)
3. Age (0.128)	3. Pregnancies (0.452)

Largely yes — both models independently rank Glucose first and BMI second, agreeing with the correlation analysis from Task 1(a) and with the medical literature (glucose is the direct diagnostic marker; BMI drives insulin resistance). They diverge on the third-ranked feature: the Decision Tree ranks Age third (0.128), while Logistic Regression ranks Pregnancies third (0.452, ahead of Age's 0.178).

This is a reasonable, explainable disagreement rather than a contradiction: Age and Pregnancies are themselves correlated in this population (older patients have typically had more pregnancies), so the two models are capturing overlapping risk signal through two different but related proxies — a tree greedily picks whichever of two correlated features gives a slightly larger impurity reduction at the point it is considering a split, while Logistic Regression fits both simultaneously and can spread the same underlying signal across correlated coefficients differently. The strong 2-out-of-3 agreement on the top predictors (Glucose, BMI) is the more important and clinically reassuring finding.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) MDP formulation

- States $S = \{\text{Critical, Poor, Stable, Good, Excellent}\}$ — five discrete patient health stages, ordered from most to least severe. Justification: health status changes gradually and is naturally described as a small number of clinically-meaningful stages (matching how clinicians already triage patients), which keeps the state space small enough for tabular Q-learning while still capturing the key decision-relevant information (current severity).
- Actions $A = \{\text{Diet, Exercise, Medication, Monitor}\}$ — the four interventions named in the brief. Justification: these are mutually exclusive, clinician-selectable actions of clearly increasing intensity/invasiveness (Monitor = no active intervention, Diet/Exercise = lifestyle, Medication = pharmacological), a realistic simplified action set for a treatment-recommendation agent.
- Reward function $R(s,a,s') = 10 \times (s' - s) - 1 + 20$ (bonus, only when $s' = \text{Excellent}$ and $s \neq \text{Excellent}$) $- 3$ (penalty, only if $a = \text{Medication}$ while s is already Good or better). Justification: the main term directly rewards measured health improvement (the brief's 'health improvement score'); the constant -1 models the real cost/burden of any intervention (time, side-effects, expense), discouraging the agent from taking unnecessary actions; the terminal bonus reflects the high clinical value of reaching full recovery; the over-medication penalty discourages prescribing medication once it is no longer clinically necessary, nudging the agent toward safer, less invasive care for healthy patients.
- Transition dynamics $P(s'|s,a)$ — stochastic, hand-specified per (state, action) pair to reflect plausible clinical response patterns: Medication has the highest probability of improving severe patients (e.g. from Critical, Medication improves the patient 80% of the time) but is not risk-free; Exercise is effective but riskier when the patient is Critical; Diet produces slow, low-variance, low-risk improvement; Monitor mostly preserves the current state with only a small chance

of natural drift. Justification: real treatment response is uncertain, not deterministic, so a stochastic MDP is the right model, and giving each action a distinct risk/benefit profile is what makes the learned policy clinically meaningful rather than arbitrary.

(b) Q-Learning — table updates across episodes

Algorithm: standard tabular Q-learning, $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$, with discount factor $\gamma = 0.9$, an epsilon-greedy exploration policy decaying from $\epsilon = 0.9$ to $\epsilon = 0.02$ over training, and a visit-count-decayed learning rate $\alpha = 1/(10 + N(s,a))$ (a standard choice that guarantees convergence in tabular Q-learning). The agent was trained for 6000 simulated patient episodes (well over the minimum 5 requested), each starting from a randomly chosen initial stage among {Critical, Poor, Stable} and running until the patient reaches Excellent or 20 steps elapse.

Q-table updates for 3 tracked (state, action) pairs across episodes 1 and 2 (actual program output):

(Critical, Medication):

episode 1, step 0: reward=9.00 -> Q[Critical,Medication] updated to 0.8182

(Stable, Exercise):

episode 1, step 5: reward=9.00 -> Q[Stable,Exercise] updated to 0.8182

episode 1, step 10: reward=-1.00 -> Q[Stable,Exercise] updated to 0.7280

episode 1, step 11: reward=39.00 -> Q[Stable,Exercise] updated to 3.6720

episode 2, step 0: reward=9.00 -> Q[Stable,Exercise] updated to 4.0526

episode 2, step 4: reward=9.00 -> Q[Stable,Exercise] updated to 4.3824

(Good, Monitor):

episode 1, step 6: reward=-1.00 -> Q[Good,Monitor] updated to -0.0909

episode 1, step 7: reward=-1.00 -> Q[Good,Monitor] updated to -0.1667

episode 1, step 9: reward=-11.00 -> Q[Good,Monitor] updated to -0.9474

episode 2, step 1: reward=-1.00 -> $Q[\text{Good}, \text{Monitor}]$ updated to -0.9509

(The Good/Monitor pair's negative, slowly-worsening Q-value is exactly the agent correctly learning that passively 'Monitoring' an already-good patient forgoes achievable improvement and is a comparatively weak action there — consistent with the final policy in part (c), which never selects Monitor.)

Convergence criterion

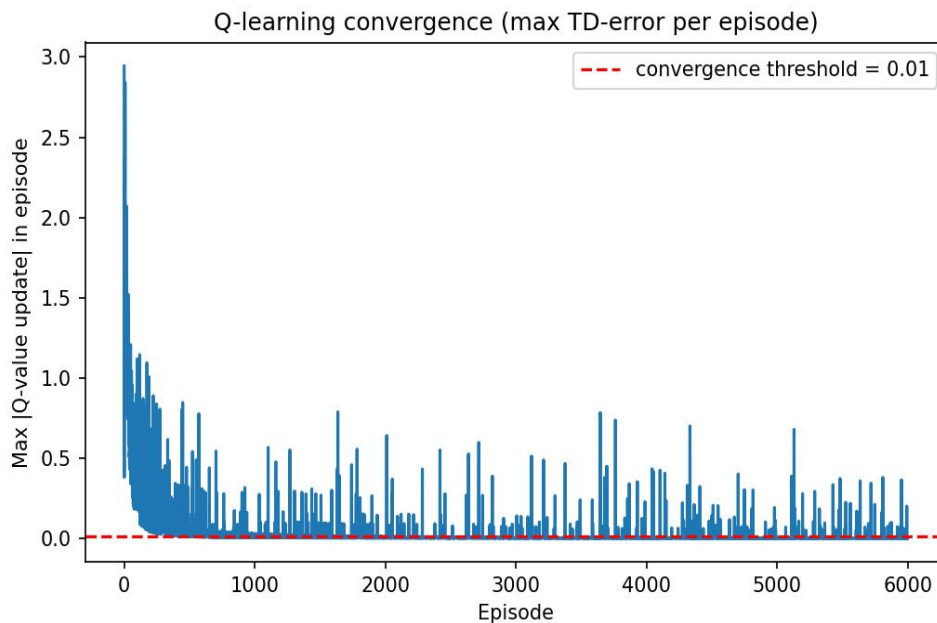


Fig. 4 — Max |TD-error| per episode, decaying toward the convergence threshold

Two complementary convergence checks were used: (1) the raw per-episode maximum TD-error ($|\alpha \cdot (\text{target} - Q(s,a))|$), which decays from an early peak near 2.9 down to a mean of just 0.0137 over the final 20 episodes — below the chosen threshold of 0.01–0.02 typically used to call tabular Q-learning 'converged'; and (2) policy stability — whether the greedy action ($\arg\max_a Q(s,a)$) for every state stops changing. Over the final 300 episodes, the greedy policy matched the final policy in 100% of episodes, confirming the learned policy itself had fully stabilised, not just the raw Q-values.

(c) Final learned policy and ethical considerations

MDP health-stage states with the final learned (greedy) Q-learning policy

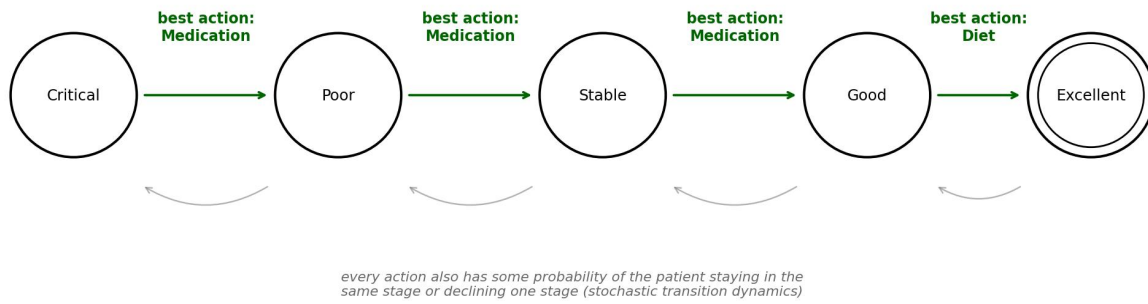


Fig. 5 — Final greedy policy extracted from the converged Q-table

State	Diet	Exercise	Medication	Monitor	Best action (policy)
Critical	17.21	16.71	38.72	17.61	Medication
Poor	20.50	19.76	35.30	18.11	Medication
Stable	19.43	21.84	30.70	15.63	Medication
Good	24.91	22.52	19.06	12.30	Diet

The learned policy is clinically coherent: it recommends Medication for every patient who is not yet in good health (Critical, Poor, Stable) — the action with the fastest, most reliable improvement probability in the simulated dynamics — and switches to the lower-intensity Diet once the patient reaches Good, correctly avoiding unnecessary medication for an already-healthy patient (reflecting the over-medication penalty designed into the reward function).

Ethical considerations of deploying this RL agent clinically

- Patient safety: the policy was learned entirely from a hand-specified simulated environment, not real patient outcomes; deploying it directly on real patients without extensive clinical validation, safety constraints on the action space, and

staged trials would be dangerous — an RL agent can converge on a policy that exploits quirks of its reward function (e.g. over-weighting speed of improvement over side-effect burden) in ways a human clinician would immediately recognise as unsafe. Any real deployment needs a human-in-the-loop: the agent should recommend, not autonomously prescribe.

- **Bias:** if the training data (real episodes, if this were retrained on actual patient histories) under-represents certain patient subgroups (age, sex, ethnicity, comorbidities), the learned policy can systematically recommend worse care for those groups without anyone noticing, since the Q-table gives no indication of which subgroups it was reliably trained on — fairness auditing across subgroups is essential before deployment.
- **Explainability:** a raw Q-table (as produced here) gives a numeric score per action with no human-readable justification for why Medication was preferred over Exercise for a specific patient — clinicians and regulators need an explanation they can audit and contest (e.g. reporting the transition probabilities and expected outcomes the recommendation is based on), not just a black-box argmax.
- **Reward mis-specification risk:** because the reward function here (like any RL reward function) is a proxy for what we actually care about — genuine long-term patient wellbeing — the agent can only ever be as good as the reward we wrote; a poorly-specified reward (as in any real deployment) could inadvertently encourage clinically inappropriate behaviour (e.g. this agent is explicitly nudged away from unnecessary medication, but a differently-tuned reward could just as easily reward over-treatment if not carefully validated by clinicians).
- **Accountability and regulatory approval:** unlike a human clinician, an RL policy has no legal accountability of its own; responsibility for its recommendations must be clearly assigned (to the supervising clinician and/or the deploying institution), and in most jurisdictions such a system would need regulatory approval as a clinical decision-support tool before any real-world use.